Лекция 15. Тесты бизнес-логики. Mock- и Stub-объекты

На прощлом уроке мы написали бизнес-логику нашего приложения (слой Service).
Тестировние Service слоя (далее для краткости будет называть его "сервис") отличается от тестирования Data слоя (далее для краткости буду неазывать его "репозиторий").
Репозиторий взаимодействует только с подключением к базе данных и нет смысла тестировать его без такого подключения.
При тестировании репозитория нам важно проверить, насколько правильно он работает с драйвером базы данных, схемой базы данных и SQL.
Сервис (как структура, в которой инкапсулирована бизнес-логика) может активно взаимодействовать со множеством других сервисов и репозиториев . При тестировании сервиса нам важно проверить два аспекта:

1. насколько правильно бизнес-логика сервиса обрабатывает переданные ей запросы и данные,
2. насколько правильно сервис взаимодействует с остальным миром.
   Первый аспект мы можем проверить с помощью изолированных юнит-тестов. В таких тестах другие сервисы и репозитории, с которыми взаимодействует тестируемый сервис, заменяются mock или stub объектами.
   Отличие между mock и stub объектами больше теоретическое, чем практическое:

- mock - это иммитация реального объекта,
- stub - это замена реального объекта.
  Для меня, эти два понятия выглядят, как братья-близнецы.
  Практическое отличие этих двух объектов заключается в том, что:
- mock создаётся с помощью библиотеки для mock-тестирования,
- stub создаётся программистом руками, путём специальной упрощённой реализации интерфейса, передаваемой в

конструктор сервиса.

На прошлом уроке мы создали employee.Service (1), который взаимодействует с API employee.Repository по интерфейсу employee.FinById (2).

service.go

```go
package employee

import "fmt"

type Service struct {
  findById FinById
}

type FinById interface {
  FindById(id int64) (employee Entity, err error)
}

func NewService(findById FinById) *Service {
  return &Service{findById: findById}
}

func (svc *Service) FindById(id int64) (Response, error) {
  var entity, err = svc.findById.FindById(id)
  if err != nil { return Response{}, fmt.Errorf("error finding employee with id %d: %w", id, err) }
  return entity.toResponse(), nil
}
```

Давайте, напишем на него тест, использующий mock-объект вместо интерфейса employee.FinById:

```go
package employee

import (
  "errors"
  "fmt"
  "github.com/stretchr/testify/assert"
  "github.com/stretchr/testify/mock"
  "testing"
  "time"
)

type MockFinById struct {
  mock.Mock
}

func (m *MockFinById) FindById(id int64) (employee Entity, err error) {
  args := m.Called(id)
  return args.Get(0).(Entity), args.Error(1)
}

func TestService(t *testing.T) {
  var a = assert.New(t)
  var findById = new(MockFinById)
  var svc = NewService(findById)

  t.Run("should return found employee", func(t *testing.T) {
    var entity = Entity{
      Id:        1,
      Name:      "John Doe",
      CreatedAt: time.Now(),
      UpdatedAt: time.Now(),
    }
    var want = entity.toResponse()

    findById.On("FindById", int64(1)).Return(entity, nil)

    var got, err = svc.FindById(1)

    a.Nil(err)
    a.Equal(want, got)
  })
```

(1) - создадим файл service_test.go
(2) - пакет с ассерт-функциями
(3) - импортируем пакет для создания моков. Если у вас этот пакет ещё не загружен в модуль, то это можно сделать командой в терминале:
go get github.com/stretchr/testify/mock
(4) - объявляем структуру мок-объекта
(5) - реализуем у мок-объекта интерфейс FinfById
(6) - общая реализация поведения мок-объекта
(7) - создаём экземпляр объекта с ассерт-функциями
(8) - создаём экземпляр мок-объекта. Делать мок репозитория общим для всех тестов имеет смысл, когда мы не проверяем состояние счётчиков мок-объекта (сколько раз он был вызван)
(9) - создаём экземпляр сервиса, который собираемся тестировать

(10) - тест, который будет проверять получение работника по его id
(11) - создаём Entity, которую должен вернуть репозиторий, и Response, который ожидаем получить от сервиса
(12) - конфигурируем поведение мок-репозитория (при вызове метода FindById с аргументом 1 вернуть Entity? созданную нами выше)
(13) - вызываем сервис с аргументом 1
(14) - проверяем, что сервис не вернул ошибку
(15) - проверяем, что сервис вернул нам то, что employee.Response, который мы ожилали получить

Теперь давайте, напишем тест, проверяющий поведение сервиса в случае получения ошибки от репозитория. В этом тесте мы также проверим, сколько раз сервис вызвал репозиторий:

```
44    t.Run("should return wrapped error", func(t *testing.T) {
45      var findById = new(MockFinById)                                  1
46      var svc = NewService(findById)                                   2
47      var entity = Entity{}                                            3
48      var err = errors.New("database error")                           4
49      var want = fmt.Errorf("error finding employee with id 1: %w", err)   5
50
51      findById.On("FindById", int64(1)).Return(entity, err)            6
52
53      var response, got = svc.FindById(1)                              7
54
55      a.Empty(response)                                                8
56      a.NotNil(got)                                                    9
57      a.Equal(want, got)                                               10
58      a.True(findById.AssertNumberOfCalls(t, "FindById", 1))           11
59    })
60  }
```

(1) - создаём для теста новый экземпляр мока репозитория. Мы собираемся проверить счётчик вызовов, поэтому хотим, чтобы счётчик содержал количество вызовов к репозиторию, выполненных в рамках одного нашего теста
(2) - создаём новый экземпляр сервиса (чтобы передать ему новый мок репозитория)
(3) - создаём пустую структуру employee.Entity
(4) - ошибка, которую вернёт репозиторий
(5) - ошибка, которую должен будет вернуть сервис
(6) - конфигурируем поведение мока репозитория в тесте
(7) - вызываем метод сервиса

(8) - проверяем, что получили пустой экземпляр employee.Entity
(9) - проверяем, что получили ошибку
(10) - проверяем, что сервис вернул ожидаемую ошибку
(11) - проверяем, сколько раз сервис вызывал репозиторий.
Тесты со stub-объектами отличаются только тем, что функционал stub-объекта (строки 16 - 19 на скриншоте с тестами) вам нужно будет реализовать самостоятельно, без использования mock-библиотек, исходя из потребностей ваших тестов.